GORDIAN KNOTZ TECHNOVATION

The Launch Readiness Handbook

56 checks across Security, Search Visibility, and Content

expanded with rationale, implementation guidance, and an explicit acceptance test for each item

Prepared 14 August 2026
Gordian Knotz Technovation · Immersicloud Consulting
Nairobi, Kenya

Security Hardening

18 checks · close before any public deployment

> *Source: "20 things to have Claude do before launching your app," @millee.md.*

S1 Hide API keys

Why Any key committed to a repo or shipped in a client bundle is a key an attacker already has. Service-role keys bypass row-level security entirely, so exposure here is not a minor leak — it is full database access.

How Move every secret to environment variables. Service-role and third-party API keys live in Vercel server-side env vars and EAS secrets, never prefixed NEXT_PUBLIC_ or EXPO_PUBLIC_. Only the anon key, which RLS constrains, is safe to ship client-side.

Done when Grepping the built client bundle for any service-role or provider secret key returns nothing.

S2 Purge Git secrets

Why Rotating a leaked key is necessary but not sufficient — the old key remains readable in every clone and fork of the repository forever unless history itself is rewritten.

How Use git-filter-repo or BFG Repo-Cleaner to strip the secret from every commit, force-push the rewritten history, and have every developer re-clone rather than pull.

Done when git log -S"<old-key>" returns zero results across all branches, and the old key returns 401 against the live endpoint.

S3 Use public DB key only

Why The anon key is designed to be public — it is constrained by RLS policies. The service-role key is not, and using it anywhere client-accessible defeats every other security control in the stack.

How Audit every client file for SUPABASE_SERVICE_ROLE_KEY. It belongs only in server-side API routes and Edge Functions, never in a component, a mobile bundle, or an env var prefixed for client exposure.

Done when The service-role key does not appear anywhere in the compiled client bundle for web or mobile.

S4 Enable row-level security

Why Without RLS, any authenticated user can potentially read or write any row in any table by calling the API directly, bypassing whatever your UI intends to restrict.

How Enable RLS on every table holding tenant or personal data. Write explicit SELECT, INSERT, UPDATE, and DELETE policies scoped to org_id and auth.uid() — an enabled table with no policy denies all access by default, which is safe but often mistaken for a bug.

Done when Every table with tenant data has RLS enabled and policies covering all four operations, verified with a cross-tenant read/write test that returns zero rows and rejects the write.

S5 Encrypt sensitive data

Why Personal and payment-adjacent data at rest is a liability if the database is ever compromised or a backup is exposed, independent of application-layer access controls.

How Use Supabase's built-in encryption at rest for the database, and column-level encryption (via pgsodium or application-layer encryption) for anything unusually sensitive such as national ID numbers or biometric templates.

Done when Sensitive columns are unreadable as plaintext from a raw database export or an unauthorised support query.

S6 Enforce server-side auth

Why A role or permission claim read from client state can be forged by anyone with browser dev tools. Trusting it for an authorization decision is equivalent to having no authorization at all.

How Every privileged action re-checks the caller's role and org membership server-side, against the JWT Supabase issued, not against a value passed in the request body or read from local storage.

Done when Manually editing a client-side role field and resubmitting a privileged request is rejected server-side.

S7 Lock record access

Why Users should only ever see and modify their own data or data their role explicitly permits. Any gap here is a privacy incident waiting for someone to notice.

How RLS policies filter every read and write to auth.uid() or the caller's org_id, not just the UI query — the API layer must enforce it independent of whatever filter the frontend happens to apply.

Done when Calling the API directly with another user's record ID returns an empty result or a rejection, not the record.

S8 Block field tampering

Why Client-side form validation is a usability feature, not a security control. A modified request can submit any field value the API will accept.

How Validate and constrain every field server-side — role, org_id, price, or status fields especially should never be trusted from client input if they carry authorization or financial meaning.

Done when Submitting a crafted request with an unauthorized field value (for example, a self-assigned admin role) is rejected server-side.

S9 Secure session cookies

Why A session cookie readable by JavaScript or sent over plain HTTP is a session hijacking vector, especially on shared or public networks common among mobile users.

How Set HttpOnly, Secure, and SameSite=Lax (or Strict where the flow allows) on every session cookie. Confirm Supabase Auth's cookie configuration matches this in both the web and any server-rendered context.

Done when Browser dev tools confirm HttpOnly and Secure flags are set; the cookie is not accessible via document.cookie.

S10 Hash passwords

Why Storing or comparing plaintext passwords means a single database breach exposes every user's credentials, often reused across other services.

How Supabase Auth handles this by default with bcrypt — the requirement here is to confirm no custom auth path or legacy import ever stored a plaintext or weakly-hashed password.

Done when A query against the auth schema confirms every stored credential is a bcrypt hash, never plaintext or an unsalted digest.

S11 Rate limit login

Why Without a limit, credential stuffing and brute-force attacks against the login endpoint are cheap and trivial to automate.

How Use Upstash Redis with @upstash/ratelimit in middleware, keyed on IP and email together so an attacker cannot lock out a legitimate user by targeting their address alone.

Done when A sixth login attempt within the configured window returns 429; a legitimate user on a different IP is unaffected.

S12 Add bot protection

Why Automated signup and login abuse burns infrastructure quota and pollutes data with fake accounts, and it typically arrives before a human ever notices the traffic.

How Add Cloudflare Turnstile or a CAPTCHA to signup, login, and password-reset forms. Prefer Turnstile for a friction-free experience on genuine users.

Done when Automated form submission without a valid challenge token is rejected server-side.

S13 Parameterize queries

Why String-concatenated SQL is the textbook injection vector — a single unescaped input field can expose or destroy the entire database.

How Use Supabase's query builder or parameterized RPC calls exclusively. Any raw SQL, including inside Postgres functions, uses bound parameters rather than string interpolation.

Done when A search for string concatenation feeding into a SQL string anywhere in the codebase returns nothing.

S14 Validate all input

Why Unvalidated input is the root cause behind most of the other items on this list — malformed or oversized data will eventually reach a code path that assumed it was clean.

How Validate type, length, and format at the API boundary with a schema library such as Zod, rejecting anything that does not match before it reaches business logic.

Done when Malformed input at every public endpoint returns a structured 400 error rather than a server error or silent acceptance.

S15 Escape user content

Why Any user-supplied text rendered without escaping is a stored or reflected XSS vector, letting an attacker run script in another user's session.

How React escapes by default — the risk is dangerouslySetInnerHTML and any raw HTML rendering path. Audit for these and sanitize with a library such as DOMPurify wherever raw HTML is unavoidable.

Done when Submitting a script tag as user input renders as inert text everywhere it is displayed, with no dangerouslySetInnerHTML left unsanitized.

S16 Restrict file uploads

Why An unrestricted upload endpoint is a path to remote code execution, storage abuse, or malware distribution through your own domain.

How Allowlist file extensions and MIME types server-side, not just in the client picker. Enforce a size limit and scan uploads where feasible. Store uploads in a bucket with RLS scoping write access to the uploader's own path.

Done when Uploading a disallowed file type or an oversized file is rejected server-side, not just blocked in the UI.

S17 Trim API responses

Why Returning full database rows leaks columns the frontend never needed — internal flags, other users' partial data, or fields that reveal more about your schema than intended.

How Select only the columns each endpoint actually needs rather than select(*). Treat every API response as something an attacker will inspect directly, not only through the intended UI.

Done when No API response includes a field the corresponding UI does not use, verified by inspecting network responses for the top ten most-used endpoints.

S18 Add security headers

Why Without a Content Security Policy and related headers, the application has no defense-in-depth against XSS or clickjacking even if every other control is correctly implemented.

How Add a headers() block in next.config.js covering CSP, X-Frame-Options, X-Content-Type-Options, Referrer-Policy, HSTS, and Permissions-Policy. Run CSP in report-only mode for 48 hours before enforcing.

Done when securityheaders.com grades the production site A or better, and zero CSP violations appear in the console during a full smoke test.

Closing note

Several items across these three lists overlap with work already specified in the AttendPAC / Actichr Deployment and Launch Handbook — row-level security, rate limiting, security headers, and the SEO and content items in particular. Where a project already has a dedicated launch checklist, treat that document as authoritative and use this handbook as the general-purpose reference for every other property.